🎮 COMPLETE LUDO GAME DOCUMENTATION

Project Name

Realtime Ludo Game with Wallet System

📌 TECH STACK

Frontend

- React Native Expo
- Zustand
- Axios
- Socket.io Client
- Firebase OTP Authentication

Backend

- Node.js
- Express.js
- Socket.io
- MongoDB + Mongoose
- JWT Authentication
- Firebase Admin SDK

Deployment

- Backend → AWS EC2 / Render
- Database → MongoDB Atlas
- Socket Server → Same Node Server
- Frontend → Expo EAS Build

📌 APPLICATION OVERVIEW

This application is a realtime 2-player Ludo game where:

- User logs in using Firebase OTP
- User gets 500 coins by default
- User can play:
- Practice Game
- Cash Game
- Cash game uses wallet coins
- Game always starts with:

• Real player OR
• Bot opponent
• Game supports:
• Realtime gameplay
• Wallet system
• Match history
• Quick chats
• Stats system

📌 CORE FEATURES

Authentication

• Firebase OTP Login
• JWT Session
• Secure login flow

Wallet

• Default 500 coins
• Debit/credit system
• Wallet transaction history

Gameplay

• 2 player ludo
• Turn based gameplay
• Dice rolling
• Token movement
• Winner calculation

Bot System

• Auto matchmaking with bot
• Smart move system
• Server-side bot logic

Cash Game

• Coin investment
• Winner gets full pool
• Backend controlled wallet distribution

Realtime Features

- Socket connection
- Live dice updates
- Live token movement
- Live chats

📌 COMPLETE SYSTEM ARCHITECTURE

```
React Native App
        ↓
Firebase Authentication
        ↓
Node.js Backend
        ↓
Socket.io Realtime Engine
        ↓
MongoDB Database
```

📌 FRONTEND FOLDER STRUCTURE

```
frontend/
│
├── src/
│   ├── assets/
│   │
│   ├── components/
│   │   ├── common/
│   │   │   ├── AppButton.js
│   │   │   ├── AppInput.js
│   │   │   ├── Loader.js
│   │   │   ├── Modal.js
│   │   │   └── Header.js
│   │   │
│   │   ├── home/
│   │   │   ├── WalletCard.js
│   │   │   ├── StatsCard.js
│   │   │   └── GameCard.js
│   │   │
│   │   ├── game/
│   │   │   ├── LudoBoard.js
│   │   │   ├── Dice.js
│   │   │   ├── Token.js
```

```
│   │   │   ├── PlayerInfo.js
│   │   │   ├── ChatBox.js
│   │   │   ├── QuickChat.js
│   │   │   └── WinnerModal.js
│   │
│   ├── screens/
│   │   ├── SplashScreen.js
│   │   ├── LoginScreen.js
│   │   ├── OtpScreen.js
│   │   ├── HomeScreen.js
│   │   ├── CashGameScreen.js
│   │   ├── PracticeGameScreen.js
│   │   ├── GameScreen.js
│   │   ├── ResultScreen.js
│   │   ├── WalletScreen.js
│   │   └── MatchHistoryScreen.js
│   │
│   ├── services/
│   │   ├── api.js
│   │   ├── auth.service.js
│   │   ├── game.service.js
│   │   ├── wallet.service.js
│   │   └── socket.service.js
│   │
│   ├── store/
│   │   ├── authStore.js
│   │   ├── gameStore.js
│   │   └── walletStore.js
│   │
│   ├── hooks/
│   │   ├── useSocket.js
│   │   └── useAuth.js
│   │
│   ├── constants/
│   │   ├── colors.js
│   │   ├── config.js
│   │   ├── game.constants.js
│   │   └── chat.constants.js
│   │
│   ├── navigation/
│   │   ├── AppNavigator.js
│   │   └── AuthNavigator.js
│   │
│   ├── utils/
│   │   ├── helpers.js
│   │   └── validations.js
│   │
│   └── App.js
│
├── .env
```

```
├── app.json
└── package.json
```

📌 BACKEND FOLDER STRUCTURE

```
backend/
│
├── src/
│   ├── config/
│   │   ├── db.js
│   │   ├── firebase.js
│   │   └── env.js
│   │
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── game.controller.js
│   │   ├── wallet.controller.js
│   │   ├── stats.controller.js
│   │   └── chat.controller.js
│   │
│   ├── services/
│   │   ├── auth.service.js
│   │   ├── game.service.js
│   │   ├── bot.service.js
│   │   ├── wallet.service.js
│   │   └── stats.service.js
│   │
│   ├── models/
│   │   ├── user.model.js
│   │   ├── game.model.js
│   │   ├── wallet.model.js
│   │   ├── transaction.model.js
│   │   └── matchHistory.model.js
│   │
│   ├── routes/
│   │   ├── auth.routes.js
│   │   ├── game.routes.js
│   │   ├── wallet.routes.js
│   │   ├── stats.routes.js
│   │   └── chat.routes.js
│   │
│   ├── sockets/
│   │   ├── index.js
│   │   ├── game.socket.js
│   │   ├── bot.socket.js
│   │   └── chat.socket.js
```

```
│   │
│   ├── middlewares/
│   │   ├── auth.middleware.js
│   │   ├── error.middleware.js
│   │   └── validation.middleware.js
│   │
│   ├── validators/
│   │   ├── auth.validator.js
│   │   └── game.validator.js
│   │
│   ├── constants/
│   │   ├── game.constants.js
│   │   └── chat.constants.js
│   │
│   ├── utils/
│   │   ├── helpers.js
│   │   ├── generateToken.js
│   │   └── ApiResponse.js
│   │
│   ├── app.js
│   └── server.js
│
├── .env
├── package.json
└── README.md
```

📌 ENV VARIABLES

Backend .env

```
PORT=5000
MONGO_URI=YOUR_MONGO_URI
JWT_SECRET=YOUR_SECRET
FIREBASE_PROJECT_ID=PROJECT_ID
FIREBASE_PRIVATE_KEY=PRIVATE_KEY
FIREBASE_CLIENT_EMAIL=CLIENT_EMAIL
SOCKET_PORT=5001
```

Frontend .env

```
EXPO_PUBLIC_API_URL=http://localhost:5000/api/v1
EXPO_PUBLIC_SOCKET_URL=http://localhost:5000
EXPO_PUBLIC_FIREBASE_API_KEY=xxxx
```

```
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=xxxx
EXPO_PUBLIC_FIREBASE_PROJECT_ID=xxxx
```

📌 DATABASE DESIGN

User Model

```
{
  phone: String,
  firebaseUid: String,
  coins: {
    type: Number,
    default: 500
  },
  wins: {
    type: Number,
    default: 0
  },
  losses: {
    type: Number,
    default: 0
  },
  totalGames: {
    type: Number,
    default: 0
  },
  createdAt: Date
}
```

Game Model

```
{
  gameId: String,
  type: String,
  status: String,
  players: [],
  currentTurn: String,
  winner: String,
  betAmount: Number,
  boardState: Object,
  moves: [],
```

```
  createdAt: Date
}
```

Transaction Model

```
{
  userId: String,
  amount: Number,
  type: String,
  reason: String,
  createdAt: Date
}
```

Match History Model

```
{
  userId: String,
  opponentId: String,
  result: String,
  gameType: String,
  coinsWon: Number,
  createdAt: Date
}
```

📌 COMPLETE AUTH FLOW

Step 1

User enters mobile number

Step 2

Firebase sends OTP

Step 3

User verifies OTP

Step 4

Frontend gets Firebase Token

Step 5

Frontend sends token to backend

Step 6

Backend verifies token

Step 7

Backend creates JWT token

Step 8

Frontend stores JWT securely

📌 COMPLETE API DOCUMENTATION

BASE URL

```
/api/v1
```

AUTH APIs

1. Verify Firebase Login

Endpoint

POST /auth/verify

Request

```
{
  "firebaseToken": "firebase_token"
}
```

Response

```
{
  "success": true,
  "token": "jwt_token",
  "user": {
    "_id": "123",
    "coins": 500
  }
}
```

---

2. Get User Profile

Endpoint

GET /auth/profile

Headers

```
Authorization: Bearer TOKEN
```

WALLET APIs

3. Get Wallet

Endpoint

GET /wallet

---

4. Get Transaction History

Endpoint

GET /wallet/history

GAME APIs

5. Create Practice Game

Endpoint

POST /game/practice/create

---

6. Create Cash Game

Endpoint

POST /game/cash/create

Request

```
{
  "betAmount": 100
}
```

---

7. Get Game Details

Endpoint

GET /game/:gameId

---

8. Roll Dice

Endpoint

POST /game/roll-dice

---

9. Move Token

Endpoint

POST /game/move-token

---

10. End Game

Endpoint

POST /game/end

---

11. Get Player Stats

Endpoint

GET /stats

CHAT APIs

12. Send Chat

Endpoint

POST /chat/send

📌 SOCKET EVENTS

Client → Server

```
join_game
roll_dice
move_token
chat_message
leave_game
```

Server → Client

```
game_joined
dice_rolled
token_moved
chat_received
game_ended
turn_changed
```

📌 SOCKET GAME FLOW

Step 1

User joins room

Step 2

Socket room created

Step 3

Players synced

Step 4

Current turn managed

Step 5

Dice rolled server-side

Step 6

Board updated

Step 7

Winner declared

📌 GAME ENGINE RESPONSIBILITIES

Backend controls:

- Dice generation
- Turn management
- Token movement validation
- Kill detection
- Winner detection
- Wallet distribution

Frontend only displays UI.

📌 BOT SYSTEM DESIGN

Bot Responsibilities

- Auto join if no player found
- Roll dice
- Move token
- Play intelligently

Bot Priorities

1. Kill enemy token
2. Save own token
3. Reach home faster
4. Random fallback move

📌 QUICK CHAT SYSTEM

Allowed Messages

```
[
  "Well Played!",
  "Oh no!",
  "I am winning 😎",
  "Play fast!",
  "Too slow!",
  "Loser!"
]
```

These messages should be stored in constants.

📌 HOME SCREEN DESIGN

Components

- Wallet Card
- Stats Card
- Practice Game Button
- Cash Game Button
- Match History Button

Home Screen Data

```
{
  coins,
  wins,
  losses,
  totalGames
}
```

📌 GAME SCREEN DESIGN

Components

- Ludo Board
- Dice Component
- Token Component
- Turn Indicator
- Player Info
- Quick Chat
- Winner Modal

📌 LUDO BOARD SYSTEM

Responsibilities

- Draw board

- Token rendering
- Safe zones
- Home path
- Token movement animation

📌 STATE MANAGEMENT

Zustand Stores

authStore

```
user
jwtToken
login()
logout()
```

---

gameStore

```
gameData
currentTurn
boardState
updateBoard()
```

---

walletStore

```
coins
transactions
updateWallet()
```

📌 MATCHMAKING SYSTEM

Practice Game

```
User joins
→ Bot joins instantly
→ Match starts
```

Cash Game

```
User selects amount
→ Coins deducted
→ Search player
→ If no player
→ Bot joins
```

📌 CASH GAME FLOW

Example

```
Player A = 100 coins
Player B = 100 coins

Pool = 200 coins
Winner gets 200
```

📌 SECURITY RULES

Never Trust Frontend

Frontend should NEVER:

- Decide winner
- Decide dice value
- Transfer wallet
- Validate moves

Backend controls everything.

Anti Cheat Rules

- Validate every move
- Verify turn
- Verify dice value
- Verify token movement
- Verify wallet amount

JWT Protection

Protected APIs:

- Wallet
- Game
- Stats
- Chat

📌 ERROR HANDLING

Standard Response Format

```
{
  "success": false,
  "message": "Invalid move"
}
```

📌 PERFORMANCE OPTIMIZATION

Backend

- Use indexes
- Use Redis later
- Socket room optimization
- Lean mongoose queries

Frontend

- Lazy loading
- Memoized components
- Avoid unnecessary renders

📌 SCALABILITY

Future Improvements

- Multiplayer rooms
- Tournament mode
- Referral system
- Leaderboards
- Daily rewards
- In-app purchases
- Real money gateway
- Clan system
- Friends system

📌 REDIS USAGE (FUTURE)

Redis can store:

- Active rooms
- Current turns
- Live board state
- Online users

📌 TESTING STRATEGY

Backend Testing

- API testing
- Socket testing
- Wallet testing
- Bot logic testing

Frontend Testing

- UI testing
- Navigation testing
- State testing

📌 DEPLOYMENT FLOW

Backend Deployment

1. Push code to GitHub
2. Deploy on Render/AWS
3. Add environment variables
4. Connect MongoDB Atlas

Frontend Deployment

1. Build APK using EAS
2. Test on device
3. Publish app

📌 RECOMMENDED PACKAGES

Backend

```
npm i express mongoose socket.io cors dotenv jsonwebtoken firebase-admin
bcrypt
```

Frontend

```
npm i axios zustand firebase socket.io-client react-navigation
```

📌 COMPLETE UI FLOW

Splash Screen

→ Check token

Login Screen

→ Enter phone number

OTP Screen

→ Verify OTP

Home Screen

→ Choose game type

Matchmaking Screen

→ Find opponent

Game Screen

→ Play game

Result Screen

→ Show winner

📌 PROJECT DEVELOPMENT PHASES

Phase 1

Setup Backend + Database

Phase 2

Firebase Authentication

Phase 3

Wallet System

Phase 4

Socket Setup

Phase 5

Ludo Game Logic

Phase 6

Bot Logic

Phase 7

Frontend UI

Phase 8

Realtime Sync

Phase 9

Testing

Phase 10

Deployment

📌 FINAL DEVELOPMENT RECOMMENDATIONS

Use Dynamic Architecture

Everything should be:

- Config driven
- Constant based

- Environment based
- Reusable
- Scalable

Avoid Hardcoding

Never hardcode:

- API URLs
- Colors
- Wallet values
- Game constants
- Chat messages
- Firebase configs

Use Reusable Components

Examples:

- Buttons
- Cards
- Inputs
- Modals
- Dice
- Tokens

FINAL CONCLUSION

This Ludo game is a complete realtime multiplayer architecture project.

Main challenging areas:

- Realtime synchronization
- Wallet integrity
- Ludo move validation
- Bot intelligence
- Socket optimization

If implemented properly, this architecture can scale into:

- Multiplayer gaming platform
- Real cash gaming app
- Tournament ecosystem
- Competitive gaming platform